AW862XX 软件设计指南

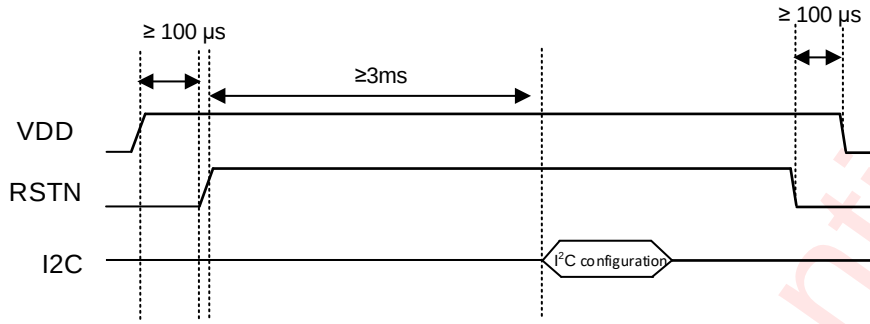
1 AW862XX 简介

AW862XX 是一款常压线性马达驱动芯片、具有基于反向电动势的 F0 检测和跟踪功能、集成了 3KB SRAM 用于用户定义波形，支持 3 个波形采样率(12K/24K/48K)。本文档旨在帮助客户快速构建 AW862XX 的驱动程序，能够支持简单的应用场景，占用较少资源。

2 软件设计

2.1 硬件复位

1. RSTN 拉低 2ms 后再拉高 8ms，完成硬件复位，参考时序图如下：



2. 参考代码：

```
HAL_GPIO_WritePin(GPIOB, GPIO_PIN_8, 0);
HAL_Delay(2);
HAL_GPIO_WritePin(GPIOB, GPIO_PIN_8, 1);
HAL_Delay(8);
```

2.2 Chipid 校验

1. AW862XX系列芯片的chipid都存放在寄存器0x00和0x64中，通过读取寄存器可以获取当前IC的chipid（AW86224与AW86225不做区分）：

IC	ADDR	Default	ADDR	Bit6	Bit0
AW86214	0x00	0x01	0x64	1	1
AW86223	0x00	0x00	0x64	0	1
AW86224	0x00	0x00	0x64	0	0
AW86225	0x00	0x00	0x64	0	0

2. 参考代码：

```
uint8_t reg[2] = {0};


haptic_nv_i2c_read(0x00, &reg[0]);
switch (reg[0]) {
case 0x00:
    haptic_nv_i2c_read(0x64, &reg[1]);
    if ((reg[1] & 0x41) == 0x01) {
        printf("aw86223 detected");
    }
    if ((reg[1] & 0x41) == 0x00) {
        printf("aw86224 or aw86225 detected");
    }
case 0x01:
    haptic_nv_i2c_read(0x64, &reg[1]);
    if ((reg[1] & 0x41) == 0x41) {
        printf("aw86214 detected");
    }
...
}
```


2.3 软件复位

1. 在识别到chipid 为四款IC 的其中之一后，可以对IC 进行软件复位操作，向0x00 寄存器写0xAA 即

可；

2. 参考代码：

```
haptic_nv_i2c_write(0x00, 0xAA);
```

2.4 芯片功能初始化

2.4.1 波形采样率

1. 通常使用 12K 波形采样率;

2. 参考代码

```
haptic_nv_i2c_write_bits(0x44, ~(3<<0), (2<<0));
```

2.4.2 保护模式

1. 直流保护功能通过寄存器 0x4E 和 0x4F 配置，PRLVL 和 PRTIME 可以默认保持，初始化为启用;

2. 参考代码

```
haptic_nv_i2c_write_bits(0x4E, ~(1<<7), (1<<7));
```

2.4.3 Gain_Bypass

1. Gain bypass 功能允许芯片在振动期间修改增益值，初始化为启用;

2. 参考代码：

```
haptic_nv_i2c_write_bits(0x49, ~(1<<6), (1<<6));
```

2.4.4 其他

```
haptic_nv_i2c_write_bits(0x2D, ~(1<<4), (1<<4));//使能 3K SRAM
haptic_nv_i2c_write_bits(0x2D, ~(1<<5), (1<<5));
haptic_nv_i2c_write_bits(0x3A, ~(3<<3), (2<<3));
haptic_nv_i2c_write_bits(0x77, ~(3<<6), (3<<6));
```

2.5 RAM 初始化

AW862XX 系列芯片拥有3KB RAM，FIFO 最大可达2KB，使用RAM 模式或RTP 模式前，需要对芯片的RAM 进行初始化配置。

2.5.1 数字模块时钟

1. 初始化 AW862XX 芯片的 RAM 之前，需要先打开数字模块的时钟，初始化完成再关闭即可；

2. 参考代码：

```
haptic_nv_i2c_write_bits(0x43, ~(1<<3), (1<<3)); //Raminit enable
HAL_Delay(1);
……                                                    //RAM config
haptic_nv_i2c_write_bits(0x43, ~(1<<3), (0<<3)); //Raminit disable
```

2.5.2 RAM基地址

1. 通过设置RAM 的基地址来划分FIFO 和RAM 空间的大小。默认使用2KB FIFO，剩余1KB 为RAM 波形存储空间;

2. 参考代码:

```
haptic_nv_i2c_write_bits(0x2D, ~(0x0F<<0), 0x08);
haptic_nv_i2c_write(0x2E, 0x00); // The RAM base addr starts from 0x0800, the FIFO size is 2KB
haptic_nv_i2c_write(0x40, 0x08);
haptic_nv_i2c_write(0x41, 0x00); // RAM pointer set to start from scratch
```

2.5.3 RAM数据初始化

1. 初始化RAM 基址后，可以将RAM 波形数据写入并存储在芯片RAM 中，并且在上电时只需写入一次RAM 波形数据；

2. 参考波形数据:

```
uint8_t aw862xx_ram_data[] = { //Default 170HZ waveform
0x55,0x08,0x11,0x09,0x76,0x09,0x77,0x0a,0x80,0x0a,0x81,0x0b,0x2e,0x0b,0x2f,0x0b,
0x61,0x00,0x0e,0x1d,0x2b,0x38,0x45,0x50,0x5b,0x63,0x6b,0x71,0x75,0x77,0x77,0x76,
0x73,0x6e,0x68,0x5f,0x56,0x4b,0x3f,0x32,0x24,0x16,0x07,0xfa,0xeb,0xdd,0xcf,0xc2,
0xb6,0xab,0xa1,0x99,0x92,0x8d,0x8a,0x89,0x89,0x8b,0x8f,0x95,0x9c,0xa5,0xaf,0xba,
0xc7,0xd4,0xe2,0xf1,0x00,0x0d,0x1c,0x2a,0x37,0x44,0x50,0x5a,0x63,0x6a,0x70,0x74,
0x77,0x77,0x76,0x73,0x6e,0x68,0x60,0x57,0x4c,0x40,0x33,0x25,0x17,0x08,0xfb,0xec,
0xde,0xd0,0xc3,0xb6,0xab,0xa2,0x99,0x93,0x8e,0x8a,0x89,0x89,0x8b,0x8f,0x94,0x9c,
0xa4,0xae,0xba,0xc6,0xd3,0xe1,0xf0,0xff,0x0c,0x1b,0x29,0x37,0x43,0x4f,0x59,0x62,
0x6a,0x70,0x74,0x77,0x77,0x76,0x73,0x6f,0x69,0x61,0x57,0x4c,0x41,0x34,0x26,0x18,
0x09,0xfb,0xed,0xde,0xd1,0xc3,0xb7,0xac,0xa2,0x9a,0x93,0x8e,0x8a,0x89,0x89,0x8b,
0x8e,0x94,0x9b,0xa4,0xae,0xb9,0xc5,0xd3,0xe1,0xef,0x00,0xf3,0xe5,0xd8,0xcb,0xbf,
0xb4,0xaa,0xa2,0x9a,0x95,0x91,0x8e,0x8e,0x8e,0x91,0x95,0x9b,0xa3,0xab,0xb5,0xc0,
0xcd,0xd9,0xe7,0xf4,0x01,0x0f,0x1d,0x2a,0x37,0x43,0x4e,0x57,0x60,0x67,0x6c,0x70,
0x72,0x72,0x71,0x6e,0x6a,0x64,0x5c,0x53,0x49,0x3e,0x32,0x25,0x17,0x0a,0xfd,0xef,
0xe1,0xd4,0xc7,0xbc,0xb1,0xa8,0x9f,0x99,0x93,0x90,0x8e,0x8e,0x8f,0x92,0x97,0x9d,
```

0xa5,0xae,0xb8,0xc4,0xd0,0xdd,0xea,0xf8,0x05,0x13,0x21,0x2e,0x3a,0x46,0x50,0x5a,
0x62,0x68,0x6d,0x71,0x72,0x72,0x71,0x6d,0x68,0x62,0x5a,0x51,0x46,0x3b,0x2e,0x21,
0x14,0x00,0xf9,0xf2,0xeb,0xe4,0xde,0xd8,0xd3,0xcf,0xcb,0xc8,0xc6,0xc5,0xc5,0xc5,
0xc7,0xc9,0xcc,0xd1,0xd5,0xdb,0xe1,0xe7,0xee,0xf5,0xfd,0x03,0x0a,0x11,0x18,0x1f,
0x25,0x2a,0x2f,0x33,0x37,0x39,0x3b,0x3b,0x3b,0x3a,0x38,0x35,0x32,0x2d,0x28,0x23,
0x1c,0x16,0x0f,0x07,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
0x00,0x00,0x07,0x16,0x24,0x32,0x3f,0x4b,0x56,0x5f,0x67,0x6e,0x73,0x76,0x77,0x77,
0x75,0x71,0x6b,0x64,0x5b,0x51,0x45,0x39,0x2b,0x1d,0x0f,0x00,0xf2,0xe4,0xd6,0xc8,
0xbc,0xb0,0xa6,0x9d,0x95,0x90,0x8b,0x89,0x89,0x8a,0x8d,0x92,0x98,0xa0,0xaa,0xb5,
0xc1,0xce,0xdb,0xea,0xf8,0x06,0x15,0x23,0x31,0x3e,0x4a,0x55,0x5f,0x67,0x6e,0x73,
0x76,0x77,0x77,0x75,0x71,0x6b,0x64,0x5b,0x51,0x46,0x39,0x2c,0x1e,0x10,0x01,0xf3,
0xe5,0xd7,0xc9,0xbc,0xb1,0xa6,0x9d,0x96,0x90,0x8c,0x89,0x89,0x8a,0x8d,0x91,0x98,
0xa0,0xa9,0xb4,0xc0,0xcd,0xda,0xe9,0xf7,0x05,0x14,0x22,0x30,0x3d,0x49,0x54,0x5e,
0x66,0x6d,0x72,0x76,0x77,0x77,0x75,0x71,0x6c,0x65,0x5c,0x52,0x47,0x3a,0x2d,0x1f,
0x11,0x00,0x0f,0x1e,0x2c,0x3a,0x47,0x53,0x5e,0x67,0x6f,0x75,0x79,0x7c,0x7c,0x7b,
0x78,0x74,0x6d,0x65,0x5b,0x50,0x44,0x37,0x29,0x1a,0x0b,0xfd,0xee,0xdf,0xd0,0xc3,
0xb6,0xaa,0xa0,0x97,0x90,0x8a,0x86,0x84,0x84,0x85,0x89,0x8e,0x95,0x9d,0xa7,0xb3,
0xbf,0xcd,0xdb,0xea,0xf9,0x07,0x16,0x25,0x33,0x41,0x4d,0x59,0x63,0x6b,0x72,0x77,
0x7b,0x7c,0x7c,0x7a,0x76,0x70,0x69,0x60,0x56,0x4a,0x3d,0x30,0x21,0x13,0x00,0xfa,
0xf4,0xee,0xe9,0xe4,0xdf,0xdb,0xd7,0xd4,0xd1,0xd0,0xcf,0xcf,0xcf,0xd0,0xd2,0xd5,
0xd9,0xdd,0xe1,0xe6,0xeb,0xf1,0xf7,0xfd,0x02,0x08,0x0e,0x14,0x1a,0x1f,0x23,0x27,
0x2b,0x2d,0x30,0x31,0x31,0x31,0x30,0x2f,0x2c,0x29,0x26,0x21,0x1d,0x17,0x12,0x0c,
0x06,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x52,0x63,0x6d,0x73,
0x78,0x7a,0x7c,0x7d,0x7d,0x7e,0x7e,0x7e,0x7e,0x7e,0x7e,0x7d,0x7c,0x7b,0x79,0x76,
0x71,0x68,0x5b,0x47,0x26,0xf2,0xc8,0xae,0x9d,0x93,0x8c,0x88,0x86,0x84,0x83,0x83,
0x82,0x82,0x82,0x82,0x83,0x83,0x84,0x85,0x88,0x8b,0x91,0x9a,0xa9,0xc1,0xe6,0x07,
0x17,0x26,0x34,0x42,0x4f,0x5a,0x64,0x6d,0x74,0x79,0x7d,0x7e,0x7e,0x7c,0x78,0x72,

```
0x6a,0x61,0x56,0x4a,0x3d,0x2f,0x20,0x11,0x00,0x0d,0x1a,0x27,0x33,0x3f,0x49,0x53,
0x5b,0x62,0x67,0x6b,0x6d,0x6d,0x6c,0x6a,0x65,0x5f,0x58,0x4f,0x46,0x3b,0x2f,0x22,
0x15,0x08,0xf9,0xe9,0xda,0xcc,0xbe,0xb1,0xa6,0x9c,0x93,0x8c,0x87,0x83,0x82,0x82,
0x84,0x88,0x8e,0x96,0x9f,0xaa,0xb6,0xc3,0xd1,0xe0,0xef,0x40,0x53,0x5e,0x66,0x6b,
0x6e,0x6f,0x71,0x71,0x72,0x72,0x72,0x72,0x72,0x72,0x72,0x71,0x70,0x6f,0x6d,0x6a,
0x64,0x5c,0x4f,0x39,0x17,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
0x08,0x11,0x19,0x21,0x28,0x2f,0x35,0x3a,0x3e,0x41,0x44,0x45,0x45,0x45,0x43,0x40,
0x3c,0x37,0x32,0x2c,0x25,0x1d,0x15,0x0d,0x04,0xfc,0xf4,0xec,0xe4,0xdc,0xd5,0xce,
0xc9,0xc4,0xc0,0xbd,0xbb,0xbb,0xbb,0xbc,0xbe,0xc2,0xc6,0xcb,0xd1,0xd8,0xdf,0xe7,
0xef,0xf7,
};
```

3. 参考代码：

```
haptic_nv_i2c_writes(0x42, aw862xx_ram_data, sizeof(aw862xx_ram_data));
```

2.6 工作

2.6.1 振动使能

1. RAM 短振参考代码:

```
haptic_nv_i2c_write(0x0A, 0x01); //Seq1 set RAM #1 waveform
haptic_nv_i2c_write(0x0B, 0x00); // Seq2 set stop
haptic_nv_i2c_write_bits(0x12, ~(0x0F<<4), (0<<4)); //Seq1 set loop 0，0x0F is infinite loop
haptic_nv_i2c_write_bits(0x08, ~(3<<0),(0<<0));//Enter RAM mode
haptic_nv_i2c_write_bits(0x08, ~(1<<2),(0<<2));//Close auto brake
haptic_nv_i2c_write(0x07, 0x80); //Set gain 0x80
haptic_nv_i2c_write(0x09, 0x01); //Start play
```

2. RAM 长振参考代码:

```
haptic_nv_i2c_write(0x0A, 0x04); //Seq1 set RAM #4 waveform

haptic_nv_i2c_write(0x0B, 0x00); // Seq2 set stop

haptic_nv_i2c_write_bits(0x12, ~(0xF0<<0), 0xF0); //Seq1 set loop 0x0F,infinite loop

haptic_nv_i2c_write_bits(0x08, ~(3<<0),(0<<0));//Enter RAM LOOP mode

haptic_nv_i2c_write_bits(0x08, ~(1<<2),(1<<2));//Open auto brake

haptic_nv_i2c_write(0x07, 0x80); //Set gain 0x80

haptic_nv_i2c_write(0x09, 0x01); //Start play
```

2.6.2 振动停止

1. 停止播放参考代码:

```
uint8_t reg_val = 0;



haptic_nv_i2c_write(0x09, 0x02);

for (int i = 0; i < 100; i++) {

    haptic_nv_i2c_read(0x3F, &reg_val); // Read global state register

    if (reg_val == 0x00) // Equal to 0 means that the chip has entered standby

        break;

    HAL_Delay(2); // Delay 2ms }

if (reg_val != 0) {

    haptic_nv_i2c_write_bits(0x44, ~(1<<6), (1<<6)); //force to standby

    haptic_nv_i2c_write_bits(0x44, ~(1<<6), (0<<6));

}
```

2.7 F0 校准

2.7.1 F0检测

1. F0 是马达共振时的频率。当驱动频率为F0 时，马达产生的振动量最大，因此线性马达要求驱动频率与马达的固有频率一致;

2．参考代码：

```
uint8_t reg_val = 0;
uint8_t brk_gain = 0x08;
uint8_t track_margain = 0x0F;
uint8_t drv2_lvl = 0;
uint32_t f0 = 0;
uint32_t f0_pre = 1700;
uint32_t vrms = 1000;
int drv_width = 0;

haptic_nv_i2c_write(0x5A, 0x00);// Set TRIM_LRA 0
haptic_nv_i2c_write_bits(0x08, ~(3<<0), (2<<0));      //Enter CONT mode
haptic_nv_i2c_write_bits(0x18, ~(1<<3), (1<<3));      //Enable F0_DET
haptic_nv_i2c_write_bits(0x1D, ~(1<<7), (1<<7));      //Enable track_en
haptic_nv_i2c_write_bits(0x08, ~(1<<2), (1<<2));      //Enable auto brake
haptic_nv_i2c_write_bits(0x1D, ~(0x7F<<0), 0x7F);    //Config cont_drv1_lvl
drv2_lvl = (((f0_pre < 1800) ? 1809920 : 1990912) / vrms * 61000000); // Calculate cont_drv2_lvl
```

```
if (drv2_lvl > 0x7F)
    drv2_lvl = 0x7F;
haptic_nv_i2c_write(0x1E, (uint8_t)drv2_lvl);          //Config cont_drv2_lvl
haptic_nv_i2c_write(0x1F, 0x04);                       //Config cont_drv1_time
haptic_nv_i2c_write(0x20, 0x06);                       //Config cont_drv2_time
haptic_nv_i2c_write (0x22, 0x0F);                      //Config cont_track_margin
drv_width = 240000 / f0_pre - 8 - brk_gain - track_margain; // Calculate cont_drv_width
if (drv_width < 0)
    drv_width = 0;
if (drv_width > 0xFF)
    drv_width = 0xFF;
haptic_nv_i2c_write(0x1A, drv_width);                  //Config cont_drv_width
haptic_nv_i2c_write(0x09, 0x01);                       //Start play
HAL_Delay(300);                                            //Delay 300ms;
haptic_nv_i2c_read(0x25, &reg_val);                    //LRA_F0 high 8 bits
f0 = reg_val << 8;
haptic_nv_i2c_read(0x26, &reg_val);                    //LRA_F0 low 8 bits
f0 = f0 | reg_val;
f0 = 384000 * 10 / f0;                                 // Calculate F0
printf("f0 is %d\r\n", f0);
haptic_nv_i2c_write_bits(0x18, ~(1<<3), (0<<3));       //Close F0_DET
haptic_nv_i2c_write_bits(0x08, ~(1<<2), (0<<2));       //Close auto brake
```

2.7.2 F0校准

1. 由于马达F0 的偏差，需要在生产线上整机情况下校准F0，保存校准值，并在播放时将其写入芯片;

2. 首先执行 F0 检测，获得马达的实际 f0，然后对其进行校准。参考校准代码如下所示:

```
char f0_cali_lra = 0;
int f0_cali_step = 0;
uint32_t f0_pre = 1700;
uint32_t f0_cali_min = 0;
uint32_t f0_cali_max = 0;

f0_cali_min = f0_pre * 93 / 100;
f0_cali_max = f0_pre * 107 / 100;
if (f0 < f0_cali_min || f0 > f0_cali_max)
    return;
f0_cali_step = 100000 * ((int)f0 - (int)f0_pre) / ((int)f0_pre * 24);
if (f0_cali_step >= 0) { /*f0_cali_step >= 0 */
if (f0_cali_step % 10 >= 5)
    f0_cali_step = 32 + (f0_cali_step / 10 + 1);
else
    f0_cali_step = 32 + f0_cali_step / 10;
}  else  { /* f0_cali_step < 0 */
if (f0_cali_step % 10 <= -5)
    f0_cali_step = 32 + (f0_cali_step / 10 - 1);
```

```
else
    f0_cali_step = 32 + f0_cali_step / 10;
}
if (f0_cali_step > 31)
    f0_cali_lra = (char)f0_cali_step - 32;
else
    f0_cali_lra = (char)f0_cali_step + 32;
printf("f0 cali data is 0x%02x\r\n", f0_cali_lra);
haptic_nv_i2c_write(0x5A, f0_cali_lra & 0x3F); //Write calibration value
```

3 参考函数

```
//i2c read byte
int haptic_nv_i2c_read(uint8_t reg_addr, uint8_t *reg_data)
{
   if (HAL_I2C_Mem_Read(&hi2c1, (0x58 << 1), reg_addr, I2C_MEMADD_SIZE_8BIT, reg_data, 1,
      1000) == HAL_OK)
            return 0;
      return -1;
}
//i2c read bytes
int haptic_nv_i2c_reads(uint8_t reg_addr, uint8_t *reg_data, uint16_t len)
{
if (HAL_I2C_Mem_Read(&hi2c1, (0x58 << 1), reg_addr, I2C_MEMADD_SIZE_8BIT, reg_data,
  len,1000) == HAL_OK)
            return 0;
      return -1;
}
//i2c write byte
int haptic_nv_i2c_write(uint8_t reg_addr, uint8_t reg_data)
{
if (HAL_I2C_Mem_Write(&hi2c1, (0x58 << 1), reg_addr, I2C_MEMADD_SIZE_8BIT, &reg_data, 1,
   1000) == HAL_OK)
```

```
        return 0;
    return -1;
}
//i2c write bytes
int haptic_nv_i2c_writes(uint8_t reg_addr, uint8_t *reg_data, uint16_t len)
{
    if (HAL_I2C_Mem_Write(&hi2c1, (0x58 << 1), reg_addr, I2C_MEMADD_SIZE_8BIT, reg_data,
        len, 1000) == HAL_OK)
        return 0;
    return -1;
}
//i2c write bits
void haptic_nv_i2c_write_bits(uint8_t reg_addr, uint32_t mask, uint8_t reg_data)
{
   uint8_t reg_val = 0;

   haptic_nv_i2c_reads(reg_addr, &reg_val, 1);
   reg_val &= (uint8_t)mask;
   reg_val |= (reg_data & (~mask));
   haptic_nv_i2c_writes(reg_addr, &reg_val, 1);
}
```

版本记录

版本	日期	说明
V1.0	2023-02-27	初版
V1.1	2023-09-05	修改芯片型号的区分说明

免责声明

此文档中包含的信息被认为是准确、可靠的。但是，上海艾为电子技术股份有限公司（以下简称艾为）对这些信息的准确性或完整性均不作任何明示或暗示的陈述或保证，且对这些信息的使用后果不承担任何责任。

艾为保留在任何时间、没有任何通报的前提下修改本文档中发布的信息，包括但不限于产品资料和规格的权利。客户在下订单前应自行获取最新的相关信息，并验证这些信息是最新且完整的。本文档信息覆盖并取代所有先于此次公布的文档。

艾为的产品没有设计、授权或保证适用于在医疗、军事、飞行器、太空或生命支持设备中使用，或是在可合理地预估艾为产品的故障或失效将导致人身伤害、死亡或严重的财产、环境损害之场合等应用。艾为不接受在上述设备或应用中纳入或使用艾为产品而衍生的任何相关责任，若在此情况下纳入或使用艾为产品，由此而产生的任何风险和责任由客户自身承担。

此文档中对艾为任何产品应用的描述仅用于举例说明，在未经进一步测试和改进的情况下，艾为不对这些应用将会适用于特定用途而作出任何陈述或保证。

所有产品的销售都必须遵守订单确认时艾为提供的商业销售一般条款和条件。

此文档中的任何文字或表述都不能被解释或解读为艾为产品可供承诺的销售要约，也不能被解读为任何专利、版权或其他工业、知识产权等任何许可的授权、转让或暗示。

对于艾为产品技术文档的信息，仅在没有对内容进行任何篡改且带有相关授权、条件、限制和声明的情况下才允许进行复制。艾为对篡改过的文件不承担任何责任或义务。复制第三方的信息可能需要遵守额外的限制条件。

在转售艾为产品或服务时，如果对该产品或服务参数的陈述与艾为标明的参数相比存在差异或虚假成分，则艾为就产品或服务所有明示或暗示的授权将失效，且这是不公平的、欺诈性商业行为。艾为对任何此类陈述均不承担任何责任或义务。